

PRÁCTICA 4. AYUDA

1. Nociones básicas.

Para esta práctica deberás crear una nueva tabla con los datos de los usuarios y sus roles. Tú decides los atributos a almacenar en la base de datos. Los roles son: normal, moderador, gestor, root. El usuario anónimo no se almacena en la BD.

2. Validación de contraseñas

Las contraseñas en una BD no se **DEBEN** almacenar en texto plano, sino que se han de codificar. Para codificarlas en PHP podemos usar la función:

```
$contraseña = 'MiContraseñaSegura123';  
$hash = password_hash($contraseña, PASSWORD_DEFAULT);
```

El resultado se almacena en la BD.

Para su comparación con un password dado por el usuario para loguearse, habrá que pedir al usuario su contraseña, y obtener la contraseña de la BD, y compararlas en PHP con:

```
password_verify($input_password, $hash)
```

nota que, el \$input_password no estará codificado, mientras que el \$hash si.

OJO: ten en cuenta en la actualización de datos que las contraseñas que ya están codificadas no pueden descodificarse. Así que piénsalo dos veces antes de volver a traértelas a la página de JS.

3. Variables de sesión

Son variables son una forma de almacenar información sobre el usuario en el servidor durante su visita a un sitio web. Esto permite que los datos persistan entre diferentes páginas (porque se guardan en el navegador). Aquí te explico cómo funcionan:

1. Se tiene que crear la sesión para consultar las variables de sesión. Esto suele ser recomendable hacerlo una sola vez por página php en la que vayas a usar las variables de sesión, al principio de la misma.

```
session_start(); //se crea la sesión
```

2. Crear las variables de sesión de la siguiente manera;

```
$_SESSION['usuario'] = 'Juan'; //se crean todas las que necesites  
$_SESSION['rol'] = 'admin';
```

3. Comprobar si existen las variables de sesión, es igual que en otros casos, se usa `isset($_SESSION['rol'])`. Devuelve true si es correcto, si no lo es, no devuelve false, devuelve el vacío. Ten cuidado con esto.

4. Eliminar una variable con:

```
unset($_SESSION['usuario']); //para eliminar una variable
```

5. Finalizar la sesión, si quieres hacerlo, porque no vas a usar más variables de sesión, entonces, puedes usar,

```
session_destroy(); // finaliza una sesión
```

6. si quieres saber si una sesión está iniciada, puedes usar

```

    if (session_status() == PHP_SESSION_NONE) {
        session_start();
    }

```

pero esto solo sería necesario si no has arrancado la sesión.

4. Recordatorio para pasar datos entre sesiones en PHP y páginas en JS

Las sesiones son un elemento de PHP, por tanto, JavaScript no tiene forma de verlas a no ser que se las pasemos de alguna manera. Vamos a recordar algunas de las opciones que tenemos... (ojo! que puede haber más)..

- Si seguimos trabajando con twig, podemos pasar parámetros a los archivos JS en propiedades de elementos html data-XXX. Estos parámetros pueden ser un JSON, o variables sueltas. Esto está explicado en la Ayuda de la P3 con más detalle.
- Llamando a código PHP desde JS, usando FETCH y leyendo el resultado de la ejecución del archivo php. Te pongo un ejemplo en los próximos apartados.

5. Gestionar datos de sesion en PHP para poder realizar comprobaciones o pasar información de sesión a JS

Ejemplo de función para pasar un objeto JSON a través de twig a una página html.

```

function cargarSesion(){
if (session_status() == PHP_SESSION_NONE) {
    session_start();
}

if (isset($_SESSION['anonimo'])){
    if ($_SESSION['anonimo'] === true) {
        return '';
    } else {

        $usuario=[
            'id' => $_SESSION['id'],
            'nombre'=>$_SESSION['nombre'],
            'rol' => $_SESSION['rol']];
        return json_encode($usuario);
    }
} else{
    $_SESSION['anonimo'] = true;
}
}

```

NOTA: Recuerda que por seguridad, los datos que almacenes en variables locales de Javascript (con let) no podrán ser accedidas por las herramientas de desarrollo de los navegadores.

6. Ejemplo para traer los datos de la sesión a JS desde PHP con FETCH

En un archivo PHP definimos esta funcionalidad:

leerSesion.php

```
<?php
session_start();
include 'flogeo.php';
if (isset($_SESSION['anonimo']) && ($_SESSION['anonimo'] ===
true)) {
    $usuario = '';
} else {
    $usuario=[
        'id' => $_SESSION['id'],
        'nombre'=>$_SESSION['nombre'],
        'rol' => $_SESSION['rol']];
}
echo json_encode($usuario);
```

En un archivo JS definimos esta función:

funciones.js

```
async function leerDatosSesion() {
    try {
        const response = await fetch('leerSesion.php');
        if (!response.ok) {
            throw new Error('Error en la solicitud');
        }
        var usuario = await response.json();

        if (usuario instanceof Promise) {
            usuario = await usuario;
        }
        usuario= JSON.stringify(usuario,null, 2);
        //convierto un objeto JSON en cadena JS
        return JSON.parse(usuario);
    } catch (error) {
        console.error('Hubo un problema:', error);
        return false; // En caso de error, devolver false
    }
}
```

En cualquier archivo que queramos leer estos datos de la sesión, podemos llamar a la función leerDatosSesion();

```
document.addEventListener('DOMContentLoaded', async function() {  
    var usuario = await leerDatosSesion();  
    // alert(usuario);  
    //Para acceder a los elementos, ej: usuario.nombre  
});
```

-